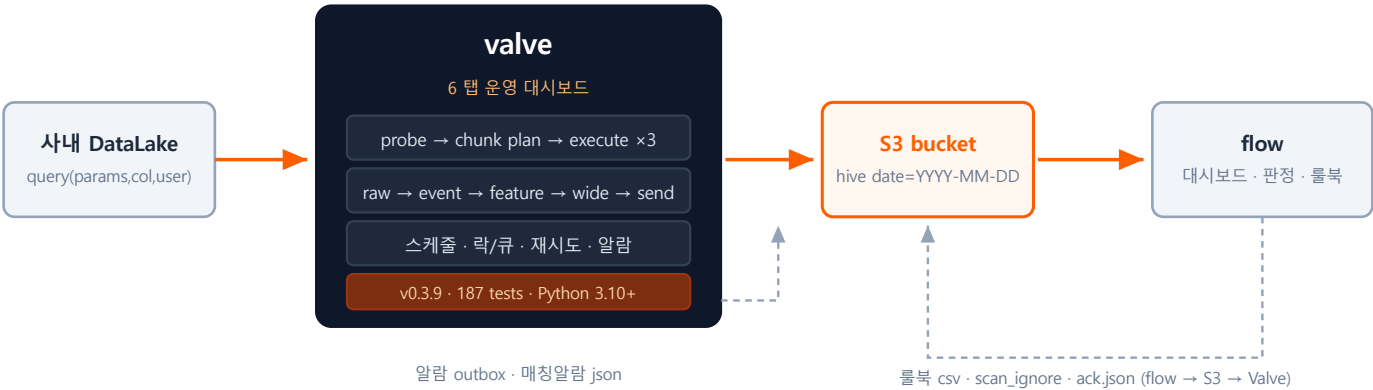


valve.

DataLake 의 수도꼭지 — 사내 API 에서 데이터를 뽑아
parquet 으로 정리하고 S3 로 흘려 **flow** 에 공급한다.

turn the valve · feed the flow



이 리포트는 2026-07-29 시점의 Valve 소스(v0.3.9, e3f9e8f)를 그대로 읽어 기능 단위로 정리한 것이다. 각 기능마다 ① 인포그래픽 ② 동작 설명 ③ 풀백·예외 처리를 함께 신는다. 풀백을 강조하는 이유는 단순하다 — Valve 가 다루는 실패는 대부분 **터지는 실패가 아니라 조용히 틀린 데이터가 나가는 실패**이고, 그걸 막는 장치들이 이 시스템의 실질적인 본체이기 때문이다.

구성

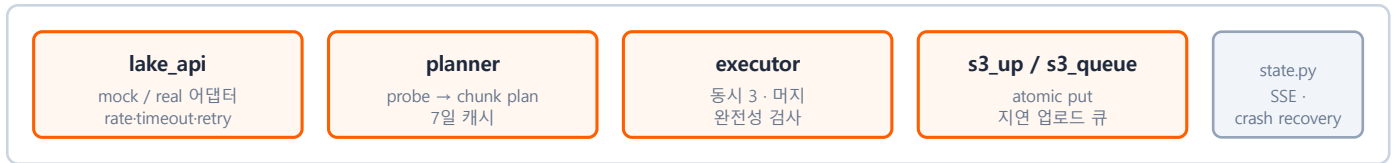
장	내용	핵심 풀백 키워드
01	시스템 전경 — 2개의 수집 경로, 5단계 파이프라인, 6탭, 산출물 트리	—
02-04	수집 계층 — Lake API 어댑터 · Probe-First 계획 · Chunk 실행/업로드	재시도·타임아웃·완전성 보류
05-09	파이프라인 — raw · event · feature · wide/send	파티션 기준 열·신선도·knob skip 보류
10-14	운영 계층 — 스케줄/금지시간대 · 실행 락/큐/취소 · 재시도 큐 · 워커 산정 · 로그/용량	안전 지점 취소·blocked·캐시
15-19	연동 계층 — 매칭알람 순환 · match_hint · S3 전송 엔진 · 설정 소유권 · FAB 스캐너	지문 발행·양방향 파일·seed-only
20-22	배포·기동 안전장치 · 풀백 총정리표 · 알려진 한계와 다음 단계	스냅샷 복원·버전 가드

표기

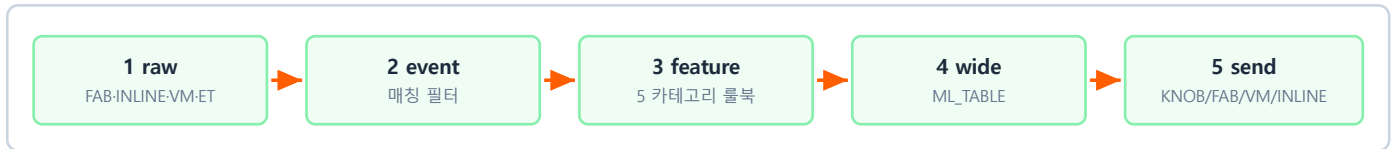
- 파일:심볼 형태의 회색 글씨는 그 설명의 **구현 위치**다. 문서와 코드가 어긋나면 코드가 맞다.
- 주황 강조**는 "이걸 모르면 반드시 한 번은 물리는" 함정이다.
- 노란 박스 = 주의 / 초록 박스 = 보증되는 성질 / 빨강 박스 = 현재 한계.

먼저 알아야 할 한 가지. Valve 에는 사내 API 로 나가는 경로가 **두 개** 있다. ① 모니터 탭의 **제품 백필**(probe → chunk → S3 직접 업로드)은 실 어댑터(`lake_api`)를 탄다. ② 알람/제품 탭이 돌리는 **파이프라인**(raw→event→feature→wide→send)의 raw 단계는 **현재 결정적 mock 합성 데이터로 동작**하며, 실 어댑터 연결이 다음 마일스톤이다 (`VERSION.json` · "다음 - 실사 내 API 연결"). 22장에 상세히 적었다.

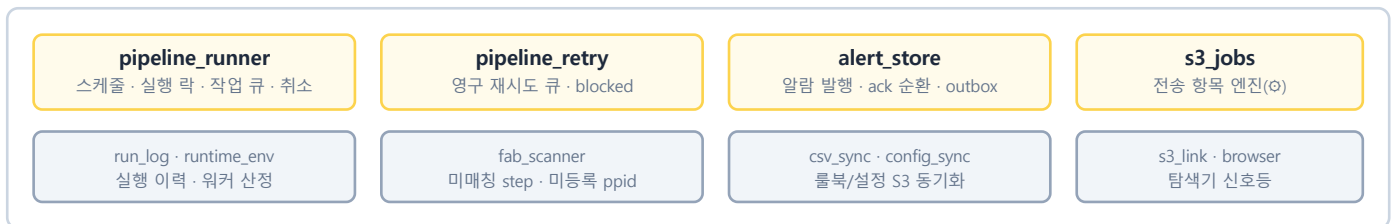
수집 계층



파이프라인



운영 · 연동



모듈 지도 — 수집 계층(위)과 파이프라인(가운데)은 서로 다른 진입점이며, 운영·연동 계층(아래)이 둘을 감싼다

화면 6탭

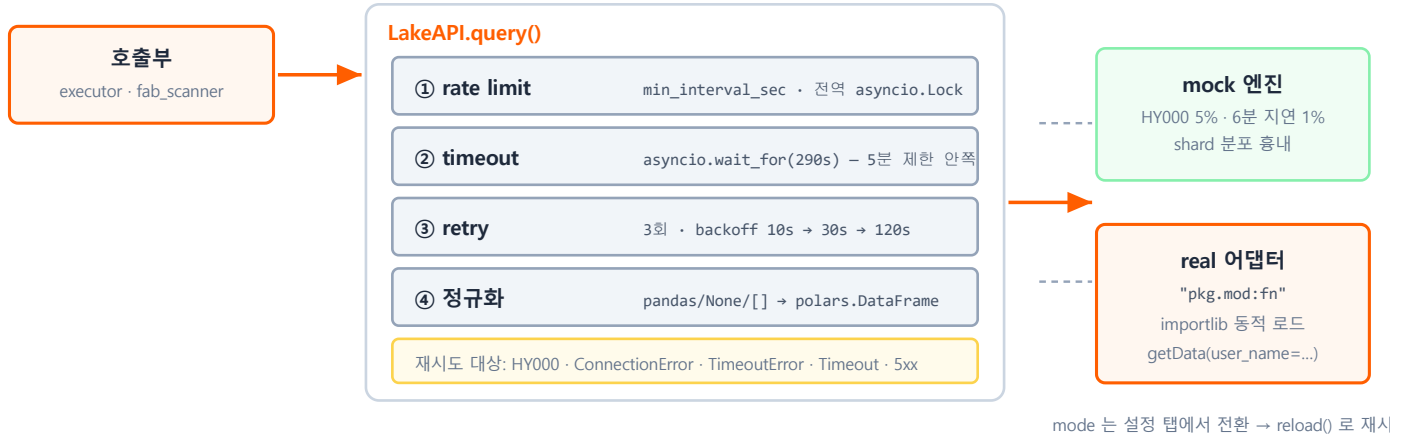
탭	주 기능	이 리포트의 해당 장
모니터	DB heatmap(제품×소스×날짜) · 진행 중 카드 · 실패 카드 · 실행 관리(전역 주기·금지 시간대·루프·제품별 주기)	05 · 10 · 14
제품	제품(vehicle) 정의 · 소스 테이블/컬럼 · params 템플릿 · 백필 enqueue	03 · 04
로그	plan/chunk/partition 이벤트 · severity pill(info/warning/critical) 필터	04 · 14
설정	<u>lake_api</u> (mode-module-user) · s3 · schedule · probe · alerts · Source types	02 · 20
탐색기	config/db/staging/outbox 파일 열람 · S3 전송 항목 · 방향 신호등 · parquet SQL 조회	17 · 18
알람	작업 큐 · 파이프라인 현황 · 실행 로그 · 매칭알람/RO ppid · 재시도 큐 · S3 업로드 풀더 · 알람 설정 3종	11 · 12 · 15 · 16

산출물 트리

경로	내용
db/1.RAWDATA_DB/{SOURCE}/{vehicle}/date=.../data.parquet	소스 원본 — 파티션 키는 데이터의 시간 컬럼 날짜
db/2.EVENT_DB/{vehicle}/{SOURCE}/date=.../data.parquet	매칭 필터 통과분 (+ <code>_meta.json</code> = 적용된 매칭 버전)
db/3.FEATURE_STORE/{vehicle}/{FEATURE}.parquet	feature 별 파일 (+ <code>_meta.json</code> = 커버 구간)
db/4.WIDE_FORM/ML_TABLE_{vehicle}.parquet	제품 1개의 전체 feature 병합
db/5.SEND_FORM/{0.KNOB,1.FAB,2.VM,3.INLINE}/	전 제품 병합 후 prefix 그룹 분리 (csv + parquet)
db/REPORTS/{vehicle}/	knob_miss · knob_skip · unmatched · scan_result · alerts_published
s3_outbox/valve-alerts/pipeline/{vehicle}.json	flow 매칭알람이 읽는 지점의 로컬 미러 (sync 대상)
staging/{product}/{SOURCE}/date=.../	백필 경로의 chunk parquet · <code>_merged.parquet</code>
logs/	pipeline_runs · jobs · pipeline_retries · s3_jobs_status/history · alerts_ack · agent_audit

app.py · feature_pipeline.db_root() · pipeline.yaml db_root (절대경로 · VALVE_DB_ROOT 우선)

사내 DataLake 의 `query(params, custom_col, user)` 하나를 감싸는 얇은 어댑터. 인증 정보는 `user_name` 하나뿐이며(키·토큰 없음), 그 위에 rate limit · timeout · 재시도 · 타입 정규화 4가지 보증을 엮는다.



폴백: retryable 이 아닌 예외는 재시도 없이 즉시 raise · retryable 목록이 비어 있으면 모든 예외가 즉시 실패
폴백: pl.from_pandas 실패(pyarrow 충돌) 시 컬럼별 list 로 dict 를 만들어 polars 재구성

mode 는 설정 탭에서 전환 → reload() 로 재시

query() 한 번의 내부 — 4가지 보증과 2가지 백엔드

Mock ↔ Real 전환

기본값은 **mock**이다. 설정 탭에서 `lake_api.mode = real` 로 바꾸고 `module` 에 `패키지.모듈:함수` 를 적으면 `importlib` 가 그 함수를 불러 쓴다. 사내 `getData` 는 keyword 시그니처라 그대로 꽂히지 않으므로 얇은 어댑터를 하나 둔다.

```
mycorp/valve_adapter.py → 설정 탭 module: mycorp.valve_adapter:query

from bigdataquery import getData

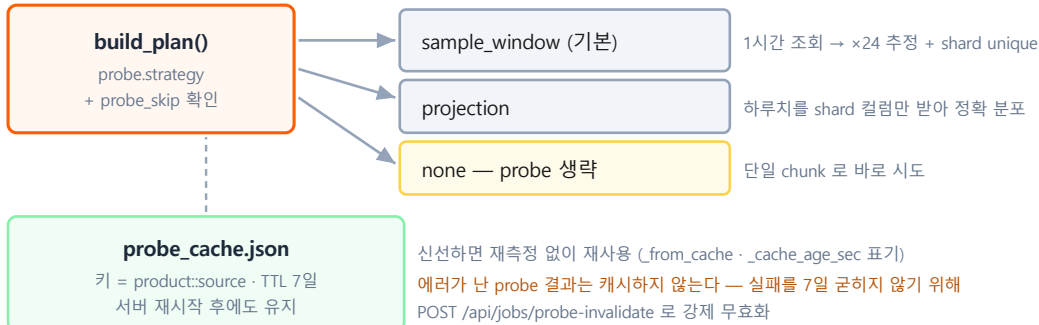
def query(params, custom_col, user):
    if custom_col:
        # 빈 리스트면 전체 컬럼 (인자 자체를 뺀다)
        return getData(params, custom_columns=custom_col, user_name=user)
    return getData(params, user_name=user)
```

인증은 `user_name` 하나뿐이다. v0.3.9 에서 쓰지도 않던 `lake_api.api_key` 를 설정·UI·매뉴얼에서 걷어냈다(구버전 설치에 남은 값은 조회 응답에서 걸러지고 저장 시 사라진다). **다시 넣지 말 것** — 없는 인증 필드는 "왜 안 되지"의 시간을 만든다.

폴백 · 예외 처리

상황	동작	근거
HY000 · 타임아웃 · 커넥션 오류	backoff 10 → 30 → 120초로 최대 3회 재시도. 마지막 예외를 그대로 올린다	query()
재시도 대상이 아닌 예외	즉시 raise — SQL 오류·권한 오류를 3배로 늘려 두드리지 않는다	_is_retryable
어댑터가 <code>None</code> · 빈 리스트 반환	빈 <code>DataFrame</code> 으로 변환. " 조회 결과 없음 "은 에러가 아니다 — 뒷단이 빈 결과로 그대로 흐른다	_to_polars
pandas → polars 변환 실패	컬럼별 <code>tolist()</code> 로 dict 를 만들어 재구성 (pyarrow 버전 충돌 회피)	_to_polars
timeout 값 초과	290초에서 끊는다 — 사내 5분 제한에 걸려 서버 쪽에서 죽는 것보다 먼저 잡는다	settings.timeout_sec
동시 호출 폭주	<code>min_interval_sec</code> 전역 락으로 최소 간격 보장 (프로세스 단위)	_wait_min_interval
설정 변경	<code>reload(settings)</code> 로 <code>__init__</code> 재실행 — mode/module/타임아웃이 재 시작 없이 반영	reload

하루치를 통째로 쓰기 전에 **1시간 샘플**로 크기와 shard 분포를 재고, 그 추정으로 chunk 를 나눈다. 측정 결과는 (제품::소스) 키로 7일 캐시돼 한 번 재면 일주일 재사용한다.



_plan_chunks — 3가지 분기

A. shard 없음 / probe 실패

단일 chunk 로 시도 → 죽으면 executor 가 기록

B. 추정 ≤ target_chunk_rows

단일 chunk (기본 target 50만 행)

C. 초과 → shard 라운드로빈

$n = \text{ceil}(\text{추정}/\text{target})$, shard 수로 상한

probe 전략 → 캐시 → chunk 분기. 어느 경로로 가도 "계획을 못 세웠으니 멈춘다"는 없다

동작

- sample_window** — `date T00:00 ~ +1h` 을 shard 키 1개 컬럼만 받아 조회하고 행 수에 `24/hours` 를 곱해 하루를 추정한다. 동시에 그 시간대에 등장한 shard(예: root_lot_id) unique 를 최대 1000개까지 담는다.
- projection** — 하루 전체를 조회하되 shard 컬럼 1~2개 + time 만 받는다. 정확하지만 비싸다.
- none** — probe 를 아예 걸지 않는다. 소스-제품 단위 `probe_skip: true` 로도 같은 경로가 된다 (probe 가 상습 실패하는 소스용).

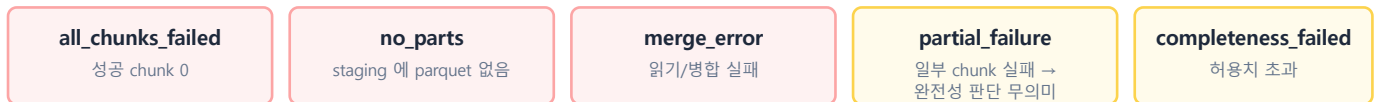
폴백 · 예외 처리

상황	동작	근거
probe 쿼리 자체가 실패	예외를 삼키고 <code>{estimated_rows:0, shards:[], error:"..."}</code> 를 반환 → 단일 chunk 로 진행 . 계획 실패가 수집 실패로 번지지 않는다	<code>_probe_sample</code>
probe 결과에 error 가 있음	캐시에 저장하지 않는다 — 실패를 7일간 재사용하면 회복이 불가능해진다	<code>build_plan</code>
그 날짜에 lot 이 하나도 없음	shard 목록이 빈 채로 정상 반환 → 단일 chunk. <code>drop_nulls()</code> 로 null shard 제거	<code>_probe_sample</code>
캐시 파일이 깨짐	<code>json.loads</code> 실패 시 빈 캐시로 시작 (지워도 다시 만들어진다 — 다음 실행이 느려질 뿐)	<code>_load_cache</code>
캐시 저장 실패(권한 등)	조용히 통과 — 계획 자체는 이미 만들어졌다	<code>_save_cache</code>
추정이 실제보다 작아 chunk 가 큼	실행 단계에서 timeout → <code>timeout_reshard</code> 로 기록하고 다음 사이클에 더 잘게 (adaptive fallback)	<code>executor</code>
shard 수 < 필요한 chunk 수	chunk 수를 shard 수로 낮춘다 (빈 chunk 를 만들지 않는다)	<code>_plan_chunks</code>

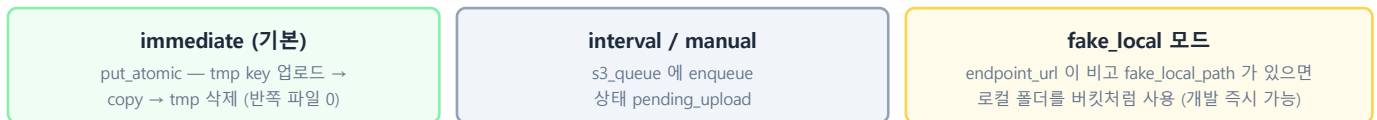
chunk 를 최대 3개까지 동시에 돌려 staging 에 떨어뜨리고, 전부 끝나면 하나로 머지해 **예상 행 수와 실제 행 수를 비교**한 뒤에야 S3 로 올린다. 어긋나면 **올리지 않고 보류**한다.



_finalize 의 5가지 중단 지점 — 하나라도 걸리면 S3 로 나가지 않는다



업로드 모드 (settings.s3.upload_mode)



업로드 실패 시 partition 상태 upload_failed + 사유 기록 · staging 의 _merged.parquet 은 남겨 탐색기에서 확인 가능
chunk 실패는 best-effort 로 통합 알람(dispatch_alert)에도 보낸다 — 알람 실패는 파이프라인을 막지 않는다

chunk 실행 → 머지 → 완전성 → 업로드. "부분 성공"을 성공으로 취급하지 않는 것이 이 단계의 핵심

폴백 · 예외 처리

상황	동작	근거
chunk 타임아웃	상태 <code>timeout_reshard</code> — 실패(failed)와 구분해 기록한다. 다음 사이클에 더 잘게 쪼갤 근거	<code>_execute_chunk</code>
shard 필터 값이 빈 목록	IN 필터를 아예 주입하지 않는다 — <code>in ()</code> 는 사내 API 가 SQL 에러를 낸다	<code>_build_params</code>
일부 chunk 만 성공	완전성 비교를 하지 않고 <code>partial_failed</code> 로 업로드 보류 + 재큐잉 대상	<code>_finalize</code>
예상 행 수가 0 (probe 실패)	완전성 검사를 건너뛴다 — 근거 없는 수치로 정상 데이터를 막지 않는다	<code>_finalize</code>
chunk 간 스키마 불일치	<code>diagonal_relaxed</code> 로 컬럼 합집합 병합 (없는 값은 null)	<code>_finalize</code>
pandas 프레임이 올라옴	polars 로 변환해 저장 — <code>pandas.to_parquet</code> 엔진 록업 버그 회피	<code>_save_staging</code>
재실행(overwrite)	plan 시작 시 해당 <code>date=</code> staging 폴더를 통째로 지우고 다시 만든다 (idempotent)	<code>run_plan</code>
boto3 미초기화 · 자격증명 없음	put/get 계열이 예외 대신 <code>False</code> / <code>None</code> 을 반환 (설정 탭에서 고칠 수 있도록 기동은 계속)	<code>s3_up</code>
fake 버킷에서 prefix 를 object 로 오인	<code>head()</code> 가 <code>is_file()</code> 로 판정 — 디렉토리를 object 로 보면 sync 비교가 깨진다 (v0.3.0 수정)	<code>s3_up.head</code>

제품(vehicle) 하나가 한 번 도는 것이 **실행 1회**다. 안에서 raw → event → feature → wide 가 순차로 돌고, 전 제품이 끝나면 마지막에 send form 이 한 번 돈다. 실행 1회 = 실행 로그 1줄.

1 · RAW

유닛 = (소스 × 1일). 병렬 실행, 전 제품 합쳐 동시 $\text{raw} \leq \text{raw_api_max}(3)$

`db/1.RAWDATA_DB/{SOURCE}/{vehicle}/date=YYYY-MM-DD/data.parquet` ← 파티션 키 = 데이터의 `tkout_time` 날짜

FAB

INLINE

VM

ET
raw만**2 · EVENT**

root_lot prefix 필터 + 소스별 매칭 필터 ($\text{FAB/VM} = \text{step_id} \cdot \text{INLINE} = \text{item_id}$)

`db/2.EVENT_DB/{vehicle}/{SOURCE}/date=.../data.parquet` + `_meta.json`(적용 매칭 버전)

매칭 버전이 바뀌면 전체 재생성
raw 가 더 새로우면 그 날짜만 재생성

3 · FEATURE

특정 기간이 아니라 그때 event DB 에 있던 전체가 대상

`db/3.FEATURE_STORE/{vehicle}/{FEATURE}.parquet` + `_meta.json`(커버 구간)

FAB_
fab.csvKNOB_
ppid_knobMASK_
mask.csvINLINE_
inline.csvVM_
vm.csv**4 · WIDE**

feature 전부를 (root_lot, wafer) 기준 left join → `ML_TABLE(vehicle).parquet` (PRODUCT 컬럼 부착)

5 · SEND

전 제품 `ML_TABLE` 병합 → prefix 그룹 분리 (0.KNOB · 1.FAB+MASK · 2.VM · 3.INLINE) csv + parquet

제품 1회 실행의 단계 흐름. 1~4 는 제품별, 5 는 전 제품 완료 후 1회

단계별 실패 파급 범위

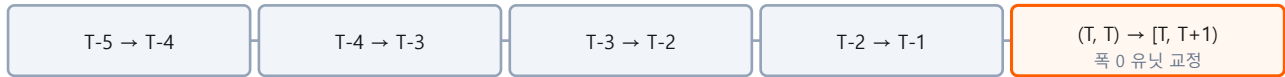
단계	실패했을 때	다음 실행에서의 회복
raw 유닛	그 (소스×날짜) 하나만 실패 — 나머지 유닛과 이후 단계는 계속 간다	재시도 큐에 남아 <code>raw_days</code> 윈도우를 벗어나도 재시도 (12장)
event	예외를 올려 그 제품 실행이 중단 (실행 로그에 critical)	다음 실행이 파티션 신선도 비교로 이어 받음
feature	feature 하나 단위로 skip 하고 사유를 남긴다 — 하나가 죽어도 나머지는 산출	원인(불복/컬럼) 수정 후 다음 실행에 자동 반영
wide	feature 가 하나도 없으면 RuntimeError	feature 단계 선행 필요
send	예외를 잡아 <code>{"error": ...}</code> 로 요약에만 남기고 실행 자체는 완료 처리	알람 탭에서 단독 재실행 가능 (<code>POST /api/pipeline/send-form</code>)

06 raw 단계 — 유닛 분할과 파티션 기준 열

feature_pipeline._run_raw_unit · _time_col ·
_write_raw_partitions

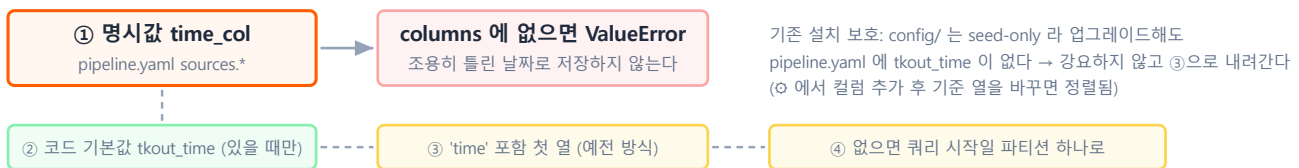
raw 는 (소스 × 하루) 단위로 쪼개 병렬 실행한다. 저장은 쿼리한 날짜가 아니라 **데이터 안의 시간 컬럼 (tkout_time) 날짜**로 파티셔닝한다 — auto report 의 daily DB 와 같은 규약이다.

유닛 분할 · 쿼리 구간 (반열림)



get_split_date_ranges(raw_days, split_days) + raw_query_window() — 항상 [from, to) 반열림이라 이웃 유닛과 겹치지 않는다
v0.3.6 이전에는 마지막 (today, today) 유닛의 폭이 0 이라 실 어댑터에서 당일 데이터가 통째로 빠질 수 있었다

파티션 기준 열 해석 순서 — _time_col()



유닛 구간과 기준 열 해석. ②의 "강요하지 않음"이 무중단 업그레이드를 보장한다

소스별 취급

소스	테이블	특징
FAB	RAW_FAB_DATA	step_id/step_desc/ppid/eqp/sleuth_order — event-feature.알람·스캔의 뿌리
INLINE	RAW_INLINE_DATA	item_id/value. v0.3.8 에서 파티션 기준을 <code>time</code> → <code>tkout_time</code> 으로 통일
VM	RAW_VM_DATA	sensor_id/residual. 위와 동일
ET	f_et_test	raw 전용 (event: false) · reformatter 기반 — config/reformatter/{vehicle}_reformatter.csv 의 CATEGORY=REAL ITEMID 만 대상. ADDP 는 파생식이라 제외

측정 시각과 track-out 시각. INLINE/VM 의 `time` (측정 시각)은 track-out 보다 늦게 찍혀 하루가 밀린다. 전 소스를 `tkout_time` 으로 통일해 둔 이유는 소스가 달라도 같은 날짜 축에 정렬하기 위해서다. **조회 컬럼에 없는 열을 기준 열로 저장하려 하면 API 가 400 으로 거부**한다 — 통과시키면 하루치가 통째로 엉뚱한 파티션에 들어간다.

폴백 · 예외 처리

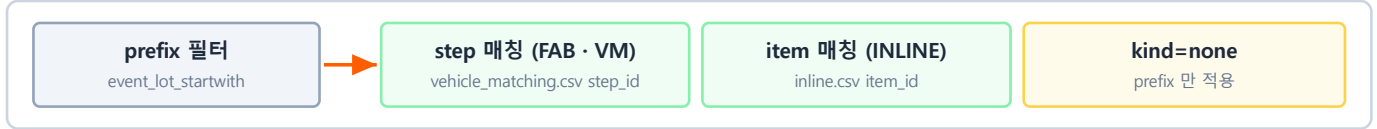
상황	동작	근거
reformatter 파일이 없는 제품	ET 를 건너뛴다 (items=None → 필터 없음이 아니라, 파일 자체가 없으면 그 소스는 대상 아님)	reformatter_items
reformatter 는 있는데 REAL 항목 0	0행 반환하고 즉시 종료 — 빈 쿼리를 사내 API 로 보내지 않는다	_run_raw_unit
어댑터가 item 필터를 무시	받은 뒤 <code>item_id.is_in(items)</code> 로 한 번 더 거른다 (실 어댑터 교체 대비 안전망)	_run_raw_unit
시간 컬럼이 비었거나 없음	쿼리 시작일 파티션 하나로 저장 — 데이터를 버리지 않는다	_write_raw_partitions
구 파일명 <code>part-000.parquet</code> 잔재	파티션 저장 시 삭제 — 같은 날짜를 두 번 읽는 중복 로드 방지	_write_partition
설정에 없는 컬럼이 응답에 섞임	<code>sources.columns</code> 교집합만 select — 스키마 드리프트가 저장 포맷을 흔들지 않는다	_run_raw_unit
쿼리 구간이 여러 날에 걸침	행마다 자기 시간 컬럼 날짜의 <code>date=</code> 로 흩어 저장 (<code>partition_by</code>)	_write_raw_partitions

현재 상태. 이 단계의 데이터 생성기는 **결정적(seed 기반) mock 합성**이다

(`_mock_fab/_mock_inline/_mock_vm/_mock_generic`). 새 소스를 추가해도 코드 수정 없이 컬럼 정의만으로 합성되도록만
들어져 있으며, 실 어댑터(`lake_api`) 연결이 다음 마일스톤이다. 22장 참조.

raw 에서 이 제품이 실제로 쓰는 행만 남긴 것이 event DB 다. 필터는 두 겹 — `root_lot_id` prefix + 소스별 매칭 테이블. 여기서 중요한 건 필터 자체보다 "언제 다시 만들 것인가"의 판정이다.

필터 2겹 (소스별 kind)



재생성 판정 — 2단 게이트



event 필터와 2단 재생성 게이트

매칭 csv ↔ event 갱신 매핑

갱신된 파일	재생성 범위
vehicle_matching.csv	FAB · VM event 전체 (적용 안 된 vehicle×소스만)
feature_rules/inline.csv	INLINE event 재생성
feature_rules/vm.csv	event 필터와 무관 — feature 단계가 매 실행 전체 재산출이라 다음 실행에 자동 반영
ppid_knob.csv · fab.csv · mask.csv	event 무관 — feature 단계에서만 쓰인다

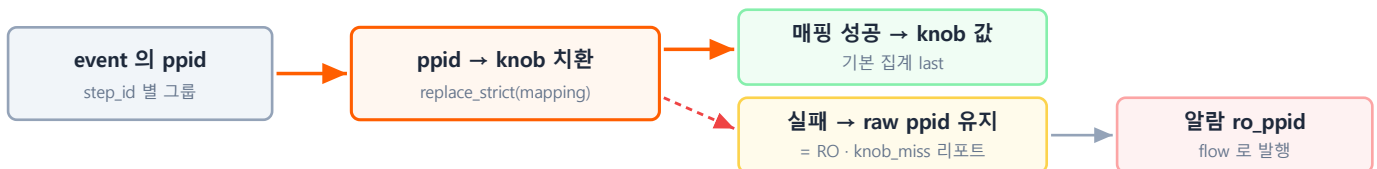
폴백 · 예외 처리

상황	동작	근거
<code>_meta.json</code> 이 깨졌거나 없음	빈 meta 로 간주 → 1회 전체 재생성 (안전한 쪽으로 판정)	run_event
mtime 조회 실패(OSError)	<code>fresh=False</code> — 즉 재생성 쪽으로 판정한다	_event_partition_fresh
event 파티션 폴더는 있는데 parquet 이 없음	fresh 아님 → 다시 만든다	_event_partition_fresh
소스를 <code>event: false</code> 로 전환	과거에 만들어진 event DB 를 정리한다 (ET)	run_event
구 레이아웃 (vehicle 바로 아래 <code>date=*</code>)	기동 시 제거 — 소스 하위 구조로 재편된 잔재 청소	run_event
FAB 컬럼이 제각각	<code>EVENT_KEEP_COLS</code> 교집합만 남기고 전부 String 캐스팅 — 날짜별 dtype 흔들림으로 concat 이 깨지지 않게	run_event
매칭 rules csv 에 id 컬럼이 없음	item_ids 빈 집합 → 그 소스 event 는 0행 (예외 대신 빈 결과)	run_event

룰북 CSV 를 읽어 wafer 한 장당 한 줄이 되도록 집계한다. 이 단계의 설계 원칙은 **"하나가 실패해도 나머지는 나간다"** — feature 단위로 예외를 잡고 사유를 `skipped` 에 남긴다.

카테고리	입력	룰북	컬럼 이름
FAB	FAB event	feature_rules/fab.csv	FAB_{step_desc}_{feature_name}
KNOB	FAB event	feature_rules/ppid_knob.csv	KNOB_{step_desc}_ppid
MASK	FAB event	feature_rules/mask.csv	MASK_{step_desc}_reticle
INLINE	INLINE event	feature_rules/inline.csv	INLINE_{item_id}_{agg}
VM	VM event	feature_rules/vm.csv	VM_{sensor_id}_residual_{agg}

KNOB — 매핑 실패는 버리지 않고 RO 로 남긴다



값을 버리면 "왜 비었지"가 되고, 조용히 채우면 틀린 값이 된다 — 그래서 원본을 남기고 알람으로 올린다.

FAB 조회 컬럼에 ppid 가 없으면 KNOB_* 전체를 skip 하고 사유를 남긴다 (sources.FAB.columns 확인).

knob skip 판정 — "확신할 수 없으면 SKIP 하지 않는다"



매칭 오류가 SKIP 으로 둔갑하지 않게

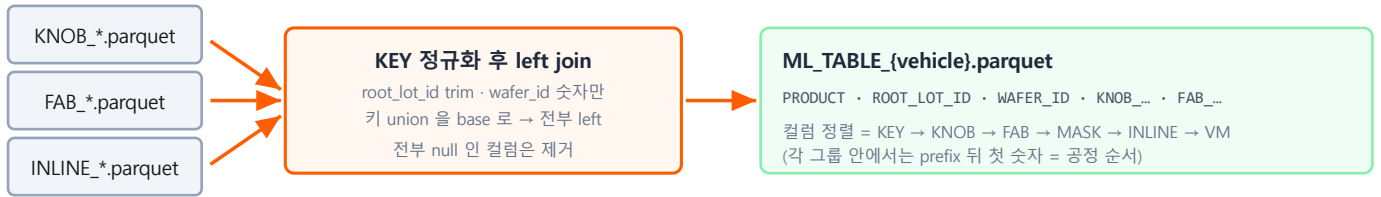
SKIP 블록의 조건 step 이 이 제품 매칭 테이블에 없으면 블록을 적용하지 않고 "vehicle_matching 확인 필요"로 리포트한다.
값 룰(category≠SKIP)의 AND 블록은 미지원으로 명시 표기한다.

비어 있음의 3분류 — pending(미도달·null 유지) / skip(뒤 step 통과) / 의심(매칭 문제·리포트)

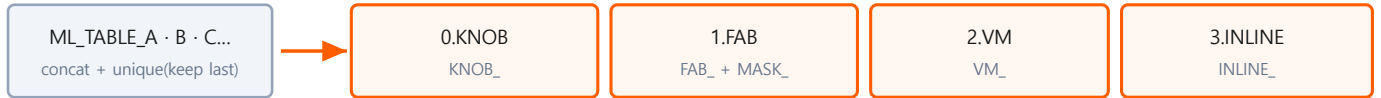
폴백 · 예외 처리

상황	동작	근거
event DB 가 없음	RuntimeError — "raw/event 단계를 먼저 실행하세요"	run_feature
룰북에 모르는 feature_name	그 feature 만 skip + "config/feature_funcs.py 에 함수 추가 가능" 안내	run_feature
집계 중 예외	해당 feature 만 skip, 사유 기록. 다른 feature 는 계속	run_feature
커스텀 함수 파일 오류	로드 오류를 skipped 로 노출. 재시작 없이 매 실행 재로드	feature_funcs
룰북 agg 와 고정 규칙 충돌	FORCED_AGG (ecual)가 이기고, 무엇이 덮였는지 agg_overrides 로 노출	run_feature
값이 전부 null·빈 문자열	그 feature parquet 을 저장하지 않는다 (빈 컬럼이 wide 를 오염시키지 않게)	_save_feature
같은 step_desc 에 step_id 여러 개	마지막 것이 이긴다(기존 동작 유지) — step_ids 는 리포트에 함께 남긴다	run_feature
커버 구간을 모름	산출 시점에 _meta.json 으로 기록 (feature parquet 에는 날짜 컬럼이 없다). 기록 없는 예전 산출물은 heatmap 이 추정으로 표시하고 툴팁에 명시	run_feature

4 · WIDE — 제품 1개



5 · SEND FORM — 전 제품



제품 안에서 합치고(wide), 제품을 넘어 합친 뒤 용도별로 나눈다(send)

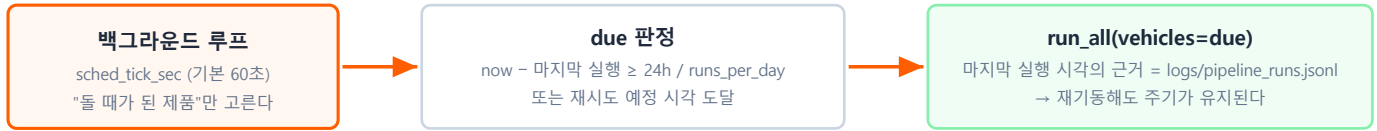
폴백 · 예외 처리

상황	동작	근거
feature 파일이 하나도 없음	RuntimeError — wide 를 만들지 않는다 (빈 ML_TABLE 로 덮어쓰지 않기 위해)	run_wide
wafer_id 표기가 제각각 (W03, 3)	숫자만 남겨 Int64 (strict=False) 로 정규화 — 실패하면 null 이 되고 조인에서 자연 제외	run_wide
같은 키가 feature 안에 중복	unique(subset=KEY) 로 먼저 접는다 — 조인 폭발 방지	run_wide
전부 null 인 feature 컬럼	제거 (KEY 컬럼은 항상 유지)	run_wide
prefix 에 숫자가 없는 컬럼	정렬 키를 inf 로 줘 그룹 맨 뒤로 — 순서가 깨지지 않는다	first_number_after
제품 간 컬럼 구성이 다름	diagonal_relaxed concat 으로 합집합, wafer 중복은 최신(keep last) 우선	run_send_form
어떤 그룹에 해당 prefix 컬럼 이 없음	파일을 만들지 않고 {"skipped": "해당 prefix 컬럼 없음"} 기록 (빈 CSV 를 flow 로 보내지 않는다)	run_send_form
실행이 취소됨	send form 을 건너뛴다 — 부분 산출물로 기존 전송본을 덮지 않는다	pipeline_runner.run_all

10 제품별 실행 주기 · 실행 금지 시간대

pipeline_runner.schedule_plan · quiet_* · _loop

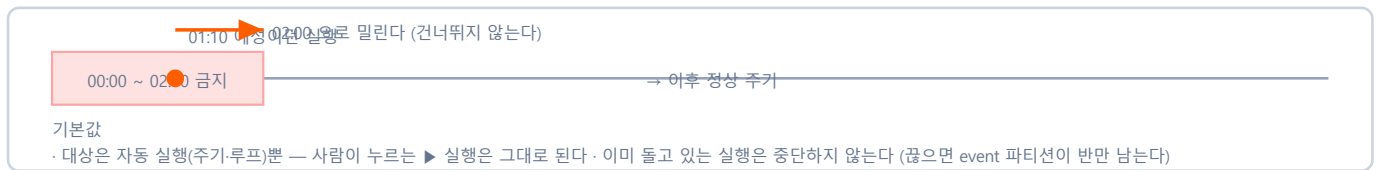
파이프라인은 제품마다 다른 주기로 돈다. 중요한 제품만 자주 돌릴 수 있고, 야간 백업·사내 API 점검 구간은 **실행 금지 시간대**로 비워 둔다.



runs_per_day 해석

N (예: 6) 24/N 시간 간격 = 4시간	0 자동 실행 제외 (수동 ▶ 는 가능)	생략 전역 runtime.interval_hours	잘못된 값 전역값으로 폴백 (예외 없음)
------------------------------	---------------------------	---------------------------------	---------------------------

실행 금지 시간대 (quiet window)



tick 은 "확인 간격"이지 주기가 아니다. 주기는 제품별 `runs_per_day` 가 정한다

폴백 · 예외 처리

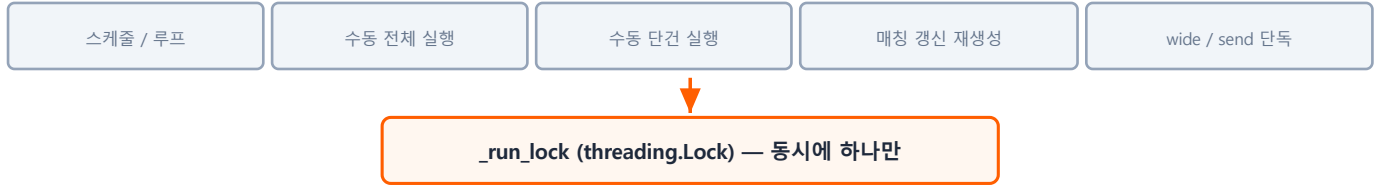
상황	동작	근거
서버 재기동	마지막 실행 시각을 실행 로그에서 복원 — 메모리에만 두면 재기동마다 전 제품이 due 로 잡혀 전량 재실행된다	run_log.last_started
실행 이력이 아예 없음(최초 기동)	즉시 due — 첫 실행을 기다리지 않는다	schedule_plan
quiet 값이 9:00 처럼 따옴표 없이 적힘	YAML 1.1 이 정수 540 으로 읽는데, 그 정수도 분으로 받아준다 (0~1440)	parse_hhmm
quiet 형식이 아예 틀림 / 시작 == 종료	구간 없음으로 간주 — 금지 시간대가 실행을 영구히 막는 사고를 만들지 않는다	quiet_window
기존 설치에 quiet 키가 없음	코드 기본값 00:00~02:00 적용. 단 빈 문자열은 "지웠다"는 뜻이므로 되살리지 않는다	quiet_cfg
quiet 중 예정 시각 도달	건너뛰지 않고 해제 시각으로 밀린다 . 화면에는 quiet_blocked 로 표시	schedule_plan
루프 모드(loop_enabled)	주기를 무시하고 연속 실행 (개발·백필용). 회차 간 loop_gap_sec 휴식	_loop
스케줄 마스터 OFF	5초마다 토글만 확인 — 켜면 즉시 반응한다	_loop
루프 안에서 예외 발생	삼키고 다음 tick 으로 — 한 번의 실패가 스케줄러를 죽이지 않는다	_loop

11 실행 락 · 작업 큐 · 안전 지점 취소

pipeline_runner_run_lock · _checkpoint · queue()

산출물을 쓰는 모든 진입점이 하나의 락을 공유한다. 그리고 그 락을 기다리는 작업까지 화면에 보여준다 — 예전에는 락 대기가 보이지 않아 "아무 일도 안 하는 것"처럼 보였다.

락을 공유하는 5개 진입점



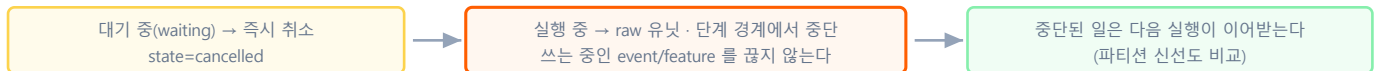
락을 공유하지 않으면 생기는 손상 3가지

- (a) 한쪽이 파티션을 지우는 사이 다른 쪽이 "이미 있음"으로 판정 → event DB 에 낱짜 구멍, 예외 없이 틀린 feature 가 나간다
- (b) 같은 data.parquet 동시 write → 잘린 parquet
- (c) _metajson 을 나중에 쓴 쪽이 이겨 stale 판정이 영영 안 됨

진입 실패 시 동작이 다르다

- run_all / 단독 작업 → 즉시 skip (다음 주기에 다시 온다)
- 매칭 갱신 재생성 → 기다린다 (기본 1800초)
건너뛰면 옛 매칭 기준 산출물이 그대로 flow 로 나가므로
대기 시간 초과취소도 last_rebuild 에 남긴다 (조용한 실패 방지)

취소 — 안전 지점에서만



락 · 큐 · 취소는 한 세트다. "무엇이 돌고 무엇이 기다리는가"가 보여야 취소가 의미를 가진다

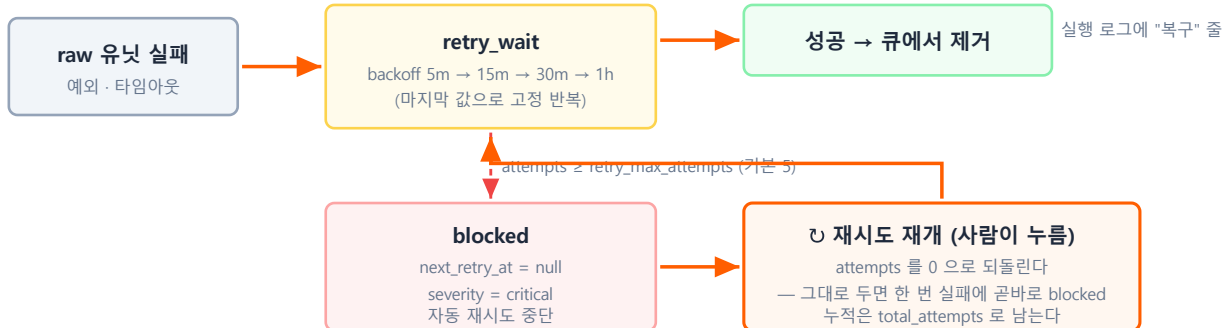
작업 큐 스냅샷 GET /api/pipeline/queue

필드	내용
running	지금 락을 쥐고 도는 작업
waiting	락을 기다리는 작업 (대기 상태에서도 취소 가능)
upcoming	제품별 다음 예정 (due · 재시도 대기 · quiet 차단 여부 포함)
recent	최근 완료 10건 (전체 보관은 최근 20건)
busy · retry	락 점유 여부 · 재시도 큐 요약

폴백 · 예외 처리

상황	동작	근거
취소 후 남은 단계	취소는 장애가 아니다 — severity=warning 으로 기록하고, 알람 발행 혹은 돌지 않는다 (옛 리포트를 내보내지 않기 위해)	run_vehicle
취소된 raw 유닛	실패로 기록하지 않는다(<code>cancelled_units</code>) — 재시도 큐를 오염시키지 않는다	_raw_guarded
취소 후 send form	건너뛴다 — 부분 산출물로 전송본을 덮지 않는다	run_all
대기 중 취소 요청	락을 잡는 순간 취소를 재확인하고 바로 놓는다 (경쟁 조건 차단)	_acquire_for
취소 불가 작업	<code>cancellable=false</code> 인 작업(wide/send 단독)은 명시적으로 거부 응답	cancel_task
제품 하나가 예외로 죽음	run_all 은 나머지 제품을 계속 처리하고 결과에 <code>error</code> 만 담는다	run_all
진행 표시가 회차 사이에 끊김	스냅샷에 <code>loop/schedule/busy/quiet</code> 를 실어 "실행 중(다음 회차 준비)"로 계속 표시 — 깜빡임 제거(v0.3.1)	snapshot

실패한 (소스 × 날짜) 유닛은 롤링 `raw_days` 윈도우를 벗어나도 큐에 남아 성공할 때까지 재시도된다. 다만 **무한 재시도는 하지 않는다** — 5회 연속 실패는 일시 장애가 아니라 사람이 고쳐야 할 문제로 본다.



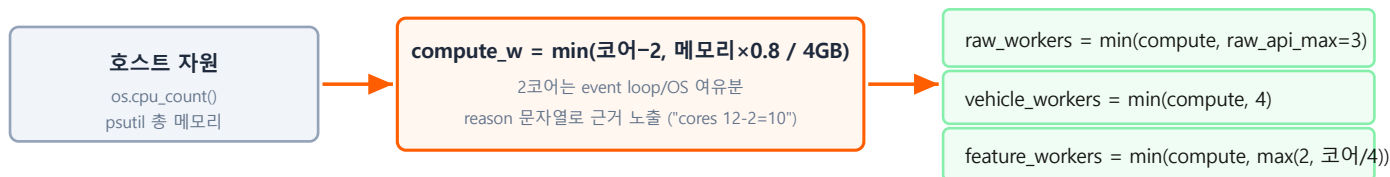
blocked 안내문: "연속 실패로 자동 재시도를 멈췄습니다 — 자격증명 만료·권한·테이블 변경 가능성을 확인하고, 해결 후 '재시도 재개'를 누르세요"
severity 승격 기준: blocked · 12시간(retry_critical_after_sec) 초과 · 5회(retry_critical_attempts) 이상 → critical / 대기 중이면 warning

재시도 상태 기계. 무한 재시도로 사내 API 를 계속 두드리는 것을 막는 것이 blocked 의 목적

폴백 · 예외 처리

상황	동작	근거
큐 파일이 BOM-깨진 JSON	<code>utf-8-sig</code> 로 읽고, 실패하면 원본을 .bad 로 보관하고 경고를 남긴다 — 여기가 조용히 비면 "놓친 유닛" 추적이 통째로 사라지는데 화면은 정상으로 보인다	<code>_load</code>
저장 중 크래시	<code>.tmp</code> 에 쓰고 <code>replace</code> — 반쪽 파일이 남지 않는다	<code>_save_locked</code>
정상 유닛과 재시도 유닛이 겹침	키 기준 중복 제거 후 추가 — 같은 유닛을 두 번 쿼리하지 않는다	<code>run_vehicle</code>
자동 실행이 꺼진 제품	재시도도 자동으로 돌지 않는다. 화면에 "자동 실행 꺼짐"으로 표시하고 수동 실행 때 함께 처리	<code>schedule_plan</code>
큐 항목의 날짜 파싱 실패	그 항목만 건너뛰다 (전체 큐를 버리지 않는다)	<code>due_units</code>
재시도 delay 설정이 이상함	정수 변환 실패 시 기본 <code>[300,900,1800,3600]</code> 로 폴백, 값은 최소 1초로 클 램프	<code>retry_delays</code>
blocked 가 섞여 있을 때 "다음 재시도"	대기 중인 것만 보고 계산한다 — blocked 는 예정 시각이 없다	<code>summary</code>

호스트의 코어와 메모리를 읽어 워커 수를 정하되, **사내 API 로 나가는 동시 요청은 따로 캡한다**. 계산 근거는 화면 (모니터 탭)에 문자열로 표시된다 — 왜 이 숫자인지 물어볼 필요가 없게.



왜 raw 만 따로 캡하나

raw 는 사내 API 풀을 쓴다 — 제품을 여러 개 동시에 돌려도 **전 제품 합쳐** 동시 raw 쿼리가 raw_api_max 를 넘지 않도록 전역 세마포어로 묶는다.
event/feature 는 API 를 쓰지 않으므로 compute 값을 그대로 쓴다. 모든 값은 pipeline.yaml runtime 에서 정수로 덮어쓸 수 있다 (sizing: config).

폴백 · 예외 처리

상황	동작	근거
psutil 미설치	POSIX <code>sysconf</code> 로 총 메모리 추정 → 그것도 없으면 코어 수만으로 산정	total_mem_gb
총 메모리를 못 구함	<code>available</code> 로 대체, 둘 다 없으면 메모리 상한을 적용하지 않는다	plan_workers
컨테이너에서 코어가 과다 보고	<code>cpu_cores</code> 를 정수로 지정해 강제 (테스트·컨테이너용)	cfg.cpu_cores
설정값이 0 이나 음수	최소 1 로 클램프 — 워커 0 으로 파이프라인이 멈추지 않는다	plan_workers

실행 로그 — 제품 × 1회 실행 = 1줄

```
[raw]      1.74s · 유닛 24 / FAB 1050 · INLINE 576 · VM 384 · ET 960 rows
[event]    1.37s / FAB: raw 0 → event 0 · 0 파티션          ← 이미 처리된 날짜는 건너뛴
[feature]  1.34s · event 19일 전체 대상 / fab 4 · knob 2 · mask 1 · inline 2 · vm 2
           knob 미변환(RO) 7건 · skip 판정 17건
           ⚠ KNOB_CONTACT_ETCH_ppid: auto skip 보류 — 뒤쪽 step 판별 불가
[wide]    0.21s · rows 705 · feature 11 → db/4.WIDE_FORM/ML_TABLE_VH_PRODA.parquet
```

트리거(`schedule` / `loop` / `manual` / `rebuild`)와 성공·실패가 함께 남고, 실패는 어느 단계까지 갔는지와 사유가 남는다. 결과 표기는 **성공** / **부분 실패** / **중단** 3단계다.

DB heatmap 의 feat 배지

• FAB feat 7 · 20일 (07-04~07-27)

feature 컬럼(parquet) 수 그 feature 가 대상으로 삼은 event 구간

기록이 없는 예전 산출물은

현재 event 기준 추정으로 표시하고
툴팁에 그렇게 적는다 — 한 번 실행하면 정확해진다

DB 사용량

제품 × 소스로 raw-event 분해
+ 전 제품 소스별 합계

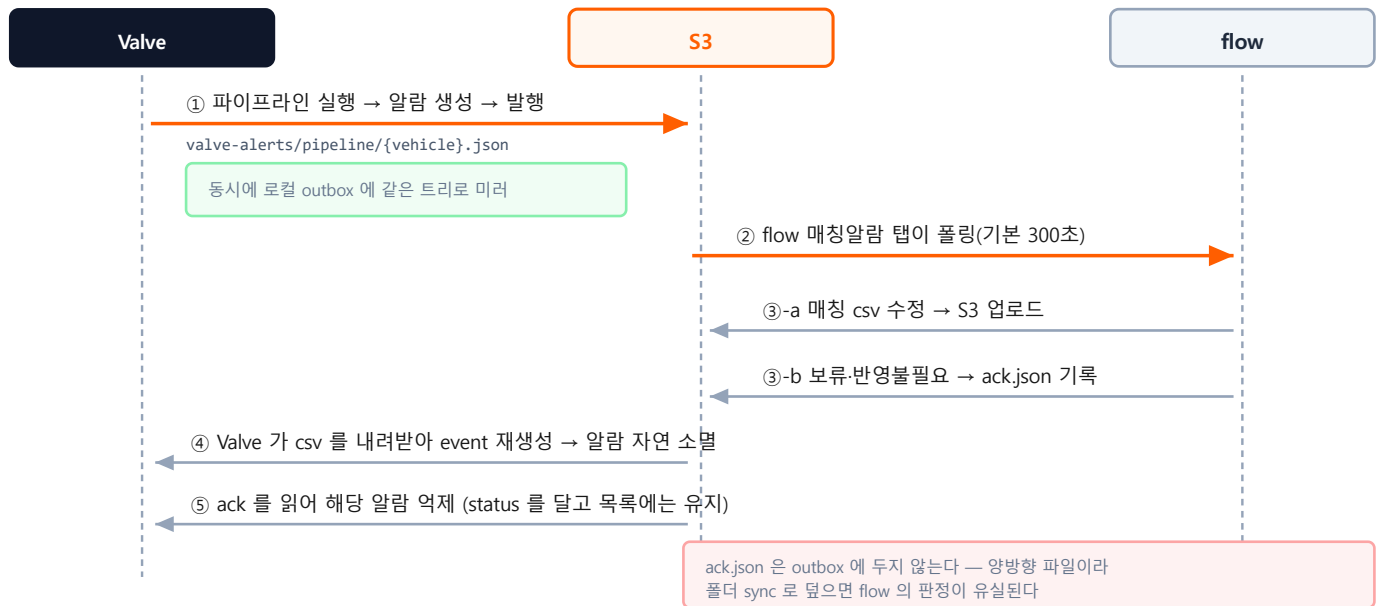
db_warn_gb(기본 40GB) 초과 시 경고
자동 삭제는 하지 않는다

전체 트리 탐색이라 10분 캐시
표기는 적응형 (KB/MB/GB)

폴백 · 예외 처리

상황	동작	근거
로그 파일 쓰기 실패	조용히 통과 — 로그 실패가 파이프라인을 멈추지 않는다	RunLog.append
로그 파일이 4MB 초과	뒤쪽(최신) 절반만 남기고 잘라낸다. 화면 표시와 스케줄 판정 모두 최신만 있으면 성립	_trim_if_needed
깨진 JSONL 줄	그 줄만 건너뛰고 나머지를 읽는다	_all
severity 가 비어 있음	<code>error</code> → critical / <code>ok=false</code> → warning / 그 외 info 로 자동 결정	append
용량 조회 중 접근 불가 폴더	<code>OSError</code> 를 건너뛰며 계속 걷는다 (심볼릭 링크는 따라가지 않음)	_dir_bytes
<code>time_col</code> 설정 오류가 있는 소스	용량 조회는 계속하고 그 소스의 기준 열만 <code>null</code> 로 — 설정 오류가 화면 전체를 막지 않는다	db_usage
<code>db_warn_gb</code> 가 0-음수 문자열	기본 40GB 로 폴백	db_warn_bytes

Valve 가 "이 step 은 매칭 테이블에 없다 / 이 ppid 는 knob 으로 못 바꿨다"를 알리면, flow 가 판정해서 룰복을 고치거나 반영불필요로 닫는다. 그 판정이 다시 Valve 로 돌아와 알람을 억제한다.



알람 1순환. 발행은 Valve, 판정은 flow, 억제는 다시 Valve

알람 id 와 발행 규칙

항목	값
미매칭 step id	um {vehicle} {step_id}
RO ppid id	ro {vehicle} {step_id} {ppid}
억제 상태	미확인예정 · 반영불필요 (지우거나 active 로 두면 다시 활성화)
발행 조건	알람 id 집합 + ack 상태의 지문(fp) 이 직전과 다를 때만. 수치(rows 등)는 파이프라인 실행 시 발행이 갱신
payload 부가	schema (현재 2) · alert_cols · delta{new,resolved} · first_seen_ts 계승
주기	alerts.s3_interval_min 기본 10분 (5~15분 권장) · 0 이면 파이프라인 실행 시에만

보증되는 성질. ① 파일은 내용이 바뀔 때만 갱신된다 — 같으면 mtime 도 건드리지 않아 aws s3 sync 가 헛돌지 않는다.
 ② outbox 는 바이트로 저장해 Windows 개행 변환으로 S3 발행본과 달라지지 않는다. ③ 폴더가 통째로 지워져도 다음 사이클에 자동 복구된다(지문이 같아도 파일이 없으면 발행).

폴백 · 예외 처리

상황	동작	근거
S3 자격증명이 없는 환경	outbox 미러는 s3_enabled 와 무관하게 항상 쓴다 — 직접 접근이 없는 환경의 유일한 경로이기 때문	publish
ack 를 S3 에서 못 읽음	로컬 캐시 <code>logs/alerts_ack.json</code> 로 폴백. 저장은 로컬 먼저, S3 는 best-effort	load_ack / set_ack
ack JSON 이 깨짐	빈 dict 로 간주 — 전 알람이 <code>active</code> 로 보인다(억제가 조용히 새는 것 보다 낫다)	load_ack
Valve 업그레이드	<code>schema</code> 버전이 지문에 섞여 있어 알람 구성이 그대로여도 한 번은 재발행 — 새 필드가 flow 로 넘어가도록	_fingerprint
발행 스냅샷 저장 실패	조용히 통과하고 발행은 계속	publish
특정 제품에서 예외	그 제품만 <code>{"error": ...}</code> 로 기록하고 나머지 제품은 계속 발행	publish_all_if_changed
주기가 0(비활성)	루프는 계속 돌되 60초마다 설정만 재확인 — 켜면 다음 사이클부터 동작(재시작 불필요)	_publish_loop
unmatched 스캔이 불가(raw 없음)	RuntimeError 를 잡아 알람 목록에서 그 부분만 제외 (RO ppid 알람은 그대로 나간다)	build

16 match_hint — function step 추천의 근거 feature_pipeline_step_match_hints → alert_store.build

flow 서버에는 FAB raw DB 가 없다. 그래서 flow 가 "이 신규 step 은 어느 function step 인가"를 LLM 에게 물을 때 쓰는 **유일한 근거가 Valve 가 알람에 실어 보내는 match_hint** 다. 한쪽만 고치면 조용히 근거 없는 추천이 된다 — 이걸 두 앱 사이의 계약이다.

step_id 는 같은 prefix-같은 자릿수 안에서 번호가 공정 순서를 뜻한다



실어 보내는 것

- 최근 N일(기본 7) FAB raw 에서 각 step 이 실제로 쓴
ppid · eqp_id · eqp_model · area 의 unique 값
- window(from~to-dates) · rows · n_lots · gap · direction
- 기간은 오늘이 아니라 데이터에 있는 날짜 기준

판단은 flow 몫

- 신규 step 의 ppid 가 이웃의 ppid 에 있으면 → 같은 function step 1순위
- 새 ppid 면 eqp_id-eqp_model-area 가 겹치는 이웃이 후보
- 한 step 은 1회만 검사하고 flow 가 결과를 기록(캐시)
- 후보 step 목록은 flow 의 Vehicle_matching.csv 가 소유

이웃으로 안 뽑는 것

- ① 아직 매칭 안 된 step (function step 이 없으니 근거가 못 된다) ② 같은 prefix 라도 자릿수가 다른 step (체계가 다르다) ③ 최근 N일 raw 에 등장하지 않은 step

번호로 이웃을 고르고, 그 이웃이 "실제로 어떻게 돌아갔는지"를 붙여 보낸다

payload 예시

```
{ "id": "um|VH_PRODA|CC955200", "type": "unmatched_step", "step_id": "CC955200",  
  "match_hint": {  
    "prefix": "CC", "number": 955200, "days": 7,  
    "window": { "from": "2026-07-20", "to": "2026-07-26", "dates": 7 },  
    "cols": [ "ppid", "eqp_id", "eqp_model", "area" ],  
    "values": { "ppid": [ "PP_SPAC_STD" ], "eqp_id": [ "CVD_02" ] }, // 신규 step  
    "neighbors": [ // 앞뒤 매칭 step  
      { "step_id": "CC955100", "step_desc": "SPACER_CVD", "direction": "prev", "gap": 100,  
        "values": { "ppid": [ "PP_SPAC_STD" ], "eqp_id": [ "CVD_02" ] }  
      }  
    ] } }
```

설정 pipeline.yaml unmatched_scan.hint · 알람 탭 ⚙ · PUT /api/pipeline/config/alert-hint

키	기본값	의미 · 클램프
enabled	true	끄면 컨텍스트를 심지 않는다 (flow 는 "근거 없음"으로 표시)
days	7	FAB raw 에서 볼 최근 날짜 파티션 수 (1~90 클램프)
neighbors	3	앞/뒤 각각의 이웃 수 (1~10)
cols	ppid, eqp_id, eqp_model, area	비교할 열. 중복 제거, 비면 기본값
value_limit	12	열 하나에 실을 unique 값 상한 (1~50)

`area` 는 조용히 빠진다. FAB raw 조회 컬럼에 없으면 hint 에서 그냥 제외된다. 쓰려면 먼저 `sources.FAB.columns` 에 추가해야 한다(알람 탭 ✕ 조회 컬럼). 기본 조회 컬럼에 넣지 않은 이유는 — 사내 테이블에 없는 열을 SELECT 에 넣으면 **FAB raw** 쿼리가 통째로 실패하기 때문이다.

폴백 · 예외 처리

상황	동작	근거
설정 키가 아예 없는 기존 설치	코드 기본값으로 그대로 동작 (<code>config/</code> 는 seed-only 라 업그레이드해도 키가 안 생긴다)	<code>alert_hint_cfg</code>
비교 열이 raw 에 하나도 없음	hint 를 만들지 않고 빈 dict 반환 → 알람에 <code>match_hint</code> 키 자체가 없다 (flow 가 "컨텍스트 없음"으로 구분)	<code>_step_match_hints</code>
<code>time_col</code> 설정 오류	기간 축소를 포기하고 전체 raw 로 계산 — 추천 컨텍스트까지 막지는 않는다	<code>_recent_raw</code>
DB 가 며칠 밀려 있음	"오늘 기준 7일"이 아니라 데이터에 있는 최근 7개 날짜 를 쓴다 — 빈 창이 나오지 않는다	<code>_recent_raw</code>
<code>step_id</code> 가 숫자로 끝나지 않음	이웃 계산을 건너뛰고 자기 값(values)만 심는다	<code>split_step_id</code>
매칭 테이블 로드 실패	<code>smap</code> 을 빈 dict 로 — 이웃이 0개가 될 뿐 알람은 정상 발행	<code>_step_match_hints</code>

검증: `Valve tests/test_pipeline.py`(hint 3건) · `flow tests/test_valve_match_hint.py`(3건)

전송은 항목(item) 단위다. "로컬 어디" ↔ "S3 어느 key" 를 짝지어 두고 방향·명령·주기를 각각 준다 (flow 의 s3_ingest 와 같은 사용 모델). 탐색기 우상단 ⚙ 에서 관리한다.

항목 하나

```
direction: download | upload
root: config | staging | db | outbox target: 상대경로
dest: S3 연결 key: S3 key/prefix
mode: sync(변경분만) | cp(항상 덮어쓰기)
interval_min: 0 = 수동 전용 enabled: true/false
```

실행 모델

- 단일 워커 큐 — 동시에 한 항목만, 나머지는 queued
- ■ 중지는 파일 경계에서 끊는다 → 이미 옮긴 파일은 남고
반쪽 파일은 생기지 않는다 (상태 cancelled)
- 진행률 5초 폴링 {done,total,current} · 이력 최근 500건
- 🔍 S3 에서 고르기 — 버킷을 계단식으로 훑어 실제 key 선택

탐색기 신호등 — 이 파일은 누가 소유하는가

↓ 파랑 — 받기

S3 → Valve. flow 가 소유하므로
올려도 다음 sync 에 뒀인다
Vehicle_matching · ppid_knob · scan_ignore

↑ 초록 — 올리기

Valve → S3
staging/ · 알람 outbox/

↕ 보라 — 양방향

기동 시 pull, 탐색기에서 push
settings · products · source_types

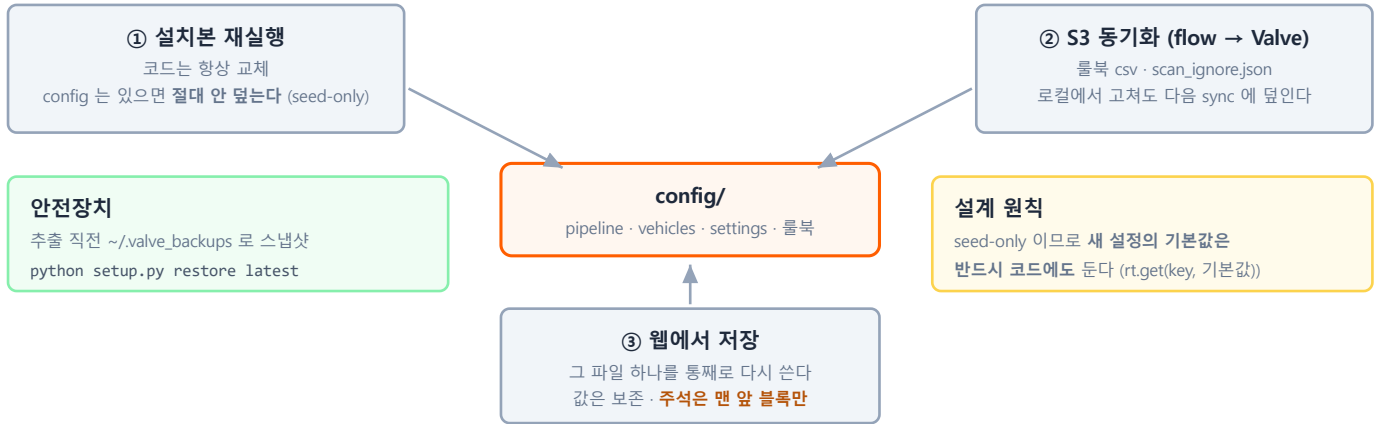
화살표 없음 — 로컬 전용

pipeline.yaml · vehicles.yaml
feature_funcs.py
점 색 = 상태(정상·대기·실패)

폴백 · 예외 처리

상황	동작	근거
최초 기동 (항목 없음)	기존 csv_sync.yaml (다운로드) · s3_transfer.yaml (업로드)에서 1회 자동 이관	migrate_if_empty
이관된 업로드 항목	enabled: false (수동 전용)로 시작 — 사고 방지. 켜는 건 사람의 판단	migrate_if_empty
알람 outbox 항목이 없음	기동 시 up_valve_alerts (sync·10분) 항목을 자동 시드 — 탐색기 db 폴더와 같은 엔진을 탄다	ensure_outbox_item
중복 실행 요청	단일 워커 큐라 순서대로 대기 — 같은 파일을 두 워커가 건드리지 않는다	s3_jobs
다운로드 항목과 legacy csv_sync 중복	다운로드 항목이 하나라도 있으면 legacy 루프를 기동하지 않는다 (이중 실행 방지)	app.py startup
설정 YAML 이 깨짐	빈 항목 목록 + 기본 플래그로 폴백 (엔진이 죽지 않는다)	S3Jobs.load
업로드 중 .tmp 파일	알람 폴더 업로드 시 제외 — 반쪽 파일이 flow 로 나가지 않는다	ensure_outbox_item

같은 `config/` 를 건드리는 주체가 셋이다. "내가 고친 게 왜 사라졌지" 는 거의 전부 여기서 갈린다.



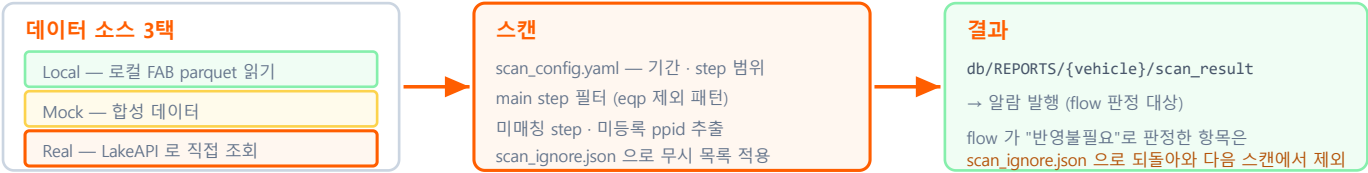
설정을 고칠 때는 "누가 이 파일의 주인인가"를 먼저 본다

주석 유실. yaml 저장은 `yaml.safe_dump` 라 주석을 전부 버리고, 복원되는 건 파일 첫 줄부터 이어지는 주석 블록뿐이다. 그래서 `pipeline.yaml` · `vehicles.yaml` 은 설명을 전부 맨 위에 모아 뒀다. **키 옆에 주석을 달면 웹에서 한 번 저장하는 순간 지워진다.** (설정 값 자체가 사라지는 일은 없다.)

기동 시 config 부트스트랩 backend/core/config_sync.py · app.py

- `settings.json` · `products.yaml` · `source_types.yaml` 을 S3 에서 pull 한다.
- 실패하면 **last_good** 으로 폴백하고 알람을 남긴다 (이벤트 루프 전이라 버퍼링했다가 startup 에서 발송).
- 동기화로 파일이 실제로 바뀌었을 때만 메모리 재로드 + `api.reload` / `s3.reload` .
- 구 포맷 `params_template({slot:{column,...}})` 은 기동 시 신 포맷으로 자동 마이그레이션 후 저장.
- `fake_local_path` 가 상대경로면 Valve ROOT 기준으로 절대화 — CWD 에 따라 다른 폴더를 쓰는 사고 방지.

파이프라인과 별개로, 지정한 기간·step 범위를 훑어 매칭 테이블에 없는 step 과 룰북에 없는 PPID 를 뽑아내는 점검 기능이다. 결과는 저장되고 알람으로 발행된다.



빈 쿼리 방어

해당 기간에 lot 이 하나도 없어도 에러가 아니다 — 스캔 쿼리는 요청한 컬럼 스키마를 보장한 빈 결과를 반환하고, 빈 결과 리포트로 정상 종료한다.

상황	동작	근거
scan_config 가 없는 제품	기본 설정으로 동작 (기간·범위 기본값)	scan_config
scan_ignore 가 비었거나 깨짐	무시 목록 없이 진행 — 무시가 조용히 적용되는 것보다 다 보이는 쪽이 안전	scan_ignore
결과 컬럼이 없는 빈 스캔	<code>_empty_result</code> 로 컬럼 구조를 갖춘 빈 결과 반환 (프론트가 깨지지 않게)	<code>_empty_result</code>

사내 배포 표준은 **self-extracting** `setup.py` 하나다 (flow 와 같은 패턴). `python setup.py` 한 번이면 소스 추출 + 의존성 설치까지 끝난다.

python setup.py

단일 파일 (약 730KB)

- ① Python 3.10 검사 → 낮으면 한글 안내 후 종료 (version/snapshots/restore 만 예외)
- ② config/ 스냅샷 → ~/.valve_backups/ ③ 코드 추출(항상 교체) · config 는 없을 때만
- ④ db_root 자동 설정 ⑤ requirements.txt 전체 pip 설치 (argv 리스트 호출)

복구

`python setup.py restore latest`
`python setup.py snapshots`

DB 루트 우선순위

VALVE_DB_ROOT (env)
> pipeline.yaml db_root > ROOT/db

C: 설치 + D: 존재

db_root 를 D:/Valve_DB 로 자동 설정
기본값('db')일 때만 — 운영자 지정값 존중

번들·쓰기에서 제외되는 것 (이중 방어)

db/ · logs/ · staging/ · s3_local/ — _build_setup.py 의 제외 목록 + setup.py 쓰기 가드. 설치본을 다시 돌려도 운영 데이터는 건드려지지 않는다.
배포 절차: VERSION.json 버전·changelog 갱신 → python _build_setup.py 로 setup.py 재빌드 → 커밋. (빌드를 빼먹으면 설치본 메타가 어긋난다)

기동 실패 대응 — 사내 이식에서 실제로 겪은 것들

증상	원인 · 해결
ModuleNotFoundError: sse_starlette	<code>pip install fastapi</code> 만으로는 부족 — <code>pip install -r requirements.txt</code> 로 전체 설치
TypeError: unsupported operand for	Python 3.9 이하. v0.3.2 부터 app.py·setup.py 가 한글 안내 후 즉시 종료한다 (<code>py -3.11</code> 처럼 버전 지정)
Invalid requirement: 'fastapi	Windows cmd 가 > 를 리다이렉션으로 해석한 것. v0.3.3 에서 pip 호출을 argv 리스트로 교체해 해결
사내망에서 pip 이 외부를 못 봄	외부 PC 에서 <code>pip download -r requirements.txt -d wheels/</code> 후 반입 → <code>--no-index --find-links wheels/</code>
업데이트했는데 화면이 그대로	정적 파일 <code>no-cache</code> 재검증(v0.3.4). 그 전 배포본은 브라우저가 구버전 app.js 를 캐시했다
aipd 패키지가 없음	브리지 라우터만 비활성화되고 Valve 본체는 정상 동작 (optional import)

런타임 반영 범위 — 재시작이 필요한가

바꾼 것	반영 시점
pipeline.yaml (runtime · sources · unmatched_scan · knob_skip)	매 호출마다 fresh 로드 — 즉시
vehicles.yaml (제품·주기)	즉시 (다음 tick)
settings.json (lake_api · s3 · alerts)	설정 API 가 in-place merge + <code>reload()</code> — 즉시
config/feature_funcs.py	실행 시점마다 다시 읽는다 — 즉시
알람 주기 / S3 항목 주기	루프가 매 사이클 설정을 다시 읽는다 — 즉시

Valve 의 폴백은 세 가지 원칙 위에 서 있다. ① **결과 없음은 실패가 아니다** — 빈 결과로 흐른다. ② **확신할 수 없으면 판단을 유보한다** — 채우지도 지우지도 않고 리포트한다. ③ **조용히 틀리느니 시끄럽게 멈춘다** — 근거 없는 산출물을 flow 로 내보내지 않는다.

계층	상황	폴백 동작
수집	HY000 · timeout · 커넥션 오류	10s → 30s → 120s 3회 재시도 후 마지막 예외 전파
	재시도 대상이 아닌 예외	즉시 실패 (사내 API 를 3배로 두드리지 않는다)
	어댑터가 None/빈 값 반환	빈 DataFrame — "데이터 없음"은 정상 경로
	pandas→polars 변환 실패	컬럼별 list 로 dict 재구성
	probe 실패	단일 chunk 로 진행 + 실패 결과는 캐시하지 않는다
업로드	일부 chunk 실패	업로드 보류(partial_failed) — 반쪽 파티션을 flow 로 보내지 않는다
	완전성 허용치 초과	업로드 보류(completeness_failed) + 재큐잉
	예상 행 수 0	완전성 검사 생략 (근거 없는 수치로 막지 않는다)
	S3 자격증명 없음	put/get 이 예외 대신 False/None — 기동은 계속, 설정 탭에서 고칠 수 있게
파이프라인	기준 열이 columns 에 없음(명시)	실패시킨다 — 하루치가 영동한 파티션에 들어가는 것보다 낫다
	기준 열이 없음(미명시)	tkout_time → 'time' 포함 첫 열 → 쿼리 시작일 순으로 폴백
	reformatter 파일 없음	그 제품의 ET 를 건너뛴다
	event_meta 손상 · mtime 조회 실패	재생성 쪽으로 판정 (안전한 방향)
	feature 하나가 실패	그 feature 만 skip + 사유 기록, 나머지는 산출
	knob 매핑 실패	raw ppid 유지(RO) + knob_miss 리포트 + 알람
판정유보	auto skip 표본·비율 미달	null 유지 + "뒤쪽 step 판별 불가" 리포트
	SKIP 블록 조건 step 이 매칭에 없음	블록 미적용 + "vehicle_matching 확인 필요"
	hint 비교 열이 raw 에 없음	그 열만 조용히 제외 · 전부 없으면 match_hint 키 자체를 안 붙인다
운영	실행 락 경합	일반 실행은 skip, 매칭 재생성은 대기(기본 1800s) 후 결과를 last_rebuild 에 기록
	실행 중 취소	안전 지점(raw 유닛·단계 경계)에서만 중단, send form 건너뛴, 다음 실행이 이어받음
	raw 유닛 실패	영구 재시도 큐 → 5회 연속 실패 시 blocked(critical) → 사람이 '재시도 재개'
	재시도 큐 파일 손상	<code>.bad</code> 로 보관 + 경고 (조용히 비우지 않는다)
	로그 쓰기 실패 · 4MB 초과	무시하고 진행 · 최신 절반만 유지
연동	S3 접근 불가	알람은 outbox 미러로 항상 남는다 (sync 만 하면 flow 갱신)
	ack 를 S3 에서 못 읽음	로컬 캐시로 폴백, 깨졌으면 전부 active 로 표시
	outbox 폴더 삭제	다음 발행 사이클에 자동 복구 (지문이 같아도 파일 없으면 발행)
	Valve 업그레이드	schema 버전이 지문에 섞여 1회 강제 재발행 — 새 필드가 flow 로 전달
	config S3 pull 실패	last_good 으로 폴백 + 알람
기동	Python 3.10 미만	한글 안내 후 종료 (restore/snapshots/version 은 허용)
	fake_local_path 준비 실패	기동을 막지 않고 오류 문자열만 보관 (설정 탭에서 수정 가능)
	aipd 미설치	브리지만 비활성화

1. 파이프라인 raw 는 아직 mock 합성이다.

`feature_pipeline._run_raw_unit` 은 `lake_api` 를 호출하지 않고 결정적 합성 데이터를 만든다. 실 어댑터를 쓰는 것은 모니터 탭의 제품 백필 경로(probe → chunk → executor)다. 즉 `lake_api.mode: real` 로 바뀌도 **파이프라인 5단계는 여전히 합성 데이터로 돈다**. VERSION.json 의 "다음 — 실사내 API 연결"이 이 작업이며, 붙는 지점은 `_run_raw_unit` 의 생성기 자리 하나다 (컬럼.파티션.필터 규약은 이미 실 데이터 기준으로 맞춰져 있다).

2. `_SUCCESS` 마커가 없다.

개별 파일은 원자적으로 쓰이지만(`put_atomic`), 파티션 폴더가 반쯤 채워진 순간에 flow 가 읽으면 일부 파일만 보게 되는 이론적 race 가 남아 있다. 실무 영향은 작지만(flow 쪽이 관대하게 fallback) 정합성 보강 여지로 남는다.

docs/flow_integration.md §6

3. 백필 경로의 S3 key prefix. executor 는 `{SOURCE}/{PRODUCT}/date=.../part-0.parquet` 로 올린다. flow 의 규약은

`1.RAWDATA_DB_{SOURCE}/...` 이므로, 한 버킷을 직접 물릴 때는 prefix 한 줄 정렬이 필요하다 (`s3.prefix` 설정 또는 key 규칙 변경). docs/flow_integration.md §3 Option A/B — 다만 이 문서는 2026-04-24 기준이라 현재 파이프라인/전송 항목 엔진 구조와는 일부 어긋난다.

4. 알람 outbox 는 "폴더를 올려주는 누군가"가 필요하다. Valve 는 `s3_outbox/valve-alerts/` 를 항상 최신으로 유지하지만, 실제 업로드는 ① `s3_jobs` 의 `up_valve_alerts` 항목(자동 시드, 10분 sync) 또는 ② 외부 `aws s3 sync` 중 하나가 담당한다. 둘 다 꺼져 있으면 flow 는 알람을 못 본다.

5. 검증 범위. `python -m pytest tests -q` 기준 187개가 모두 mock 경로를 탄다. 실 어댑터·실 S3 는 테스트 대상이 아니다. 시계에 의존하는 기본값(실행 금지 시간대 등)은 `pipe` fixture 에서 꺼야 새벽에 도는 테스트가 깨지지 않는다.

우선순위 제안

순위	할 일	이유
1	<code>_run_raw_unit</code> 에 실 어댑터 경로 연결 (+ mock 은 개발 모드로 유지)	파이프라인 전체가 실데이터로 돌기 시작하는 유일한 관문
2	파티션 완료 마커(<code>_SUCCESS</code>) 쓰기 + flow 쪽 체크	남은 정합성 구멍 하나
3	백필 경로 S3 key 를 flow 규약으로 정렬	중간 동기화 없이 바로 물릴 수 있게
4	<code>docs/flow_integration.md</code> 를 현재 구조(v0.3.9)로 갱신	2026-04-24 기준이라 지금과 어긋난다

자주 만지는 설정 키

키	기본값	의미
runtime.raw_days / split_days	5 / 1	한 번에 조회할 일수 · 유닛 분할 단위
runtime.raw_api_max	3	전 제품 합친 동시 raw 쿼리 상한
runtime.interval_hours	6	<code>runs_per_day</code> 없는 제품의 기본 주기
runtime.schedule_enabled	true	자동 실행 마스터 스위치
runtime.sched_tick_sec	60	"돌 때가 된 제품" 확인 간격
runtime.quiet_enabled / start / end	true / 00:00 / 02:00	실행 금지 시간대 (따옴표 필수)
runtime.retry_delays_sec	300-900-1800-3600	재시도 backoff (마지막 값 고정 반복)
runtime.retry_max_attempts	5	연속 실패 상한 → blocked
runtime.db_warn_gb	40	제품 DB 경고 임계 (자동 삭제 없음)
sources.<NAME>.time_col	tkout_time	파티션 기준 열 (조회 컬럼 안에서만)
unmatched_scan.alert_cols	eqp_id-eqp_model-area	매칭알람에 실을 raw 열
unmatched_scan.hint.*	7일-이웃 3	function step 추천 컨텍스트
knob_skip.min_support / after_fraction	5 / 0.8	auto skip 판정 임계
lake_api.mode / module / user	mock / — / pipe-runner	어댑터 전환 · 인증은 user 하나
alerts.s3_interval_min / outbox_dir	10 / s3_outbox	알람 발행 주기 · 미러 폴더
s3.upload_mode	immediate	immediate / interval / manual

주요 API

경로	설명
GET /api/pipeline/queue · POST .../queue/cancel	작업 큐(실행·락대기·예정) · 안전 지점 취소
GET /api/pipeline/retries · POST .../retries/resume	재시도 큐 · blocked 재개
GET /api/pipeline/schedule · PUT .../{vehicle}	제품별 주기 조회·저장 (0=제외, null=전역)
GET /api/pipeline/runs	실행 로그 (vehicle-limit-failed_only 필터)
GET /api/pipeline/db-usage	제품 × 소스 raw-event 사용량 (10분 캐시)
GET /api/pipeline/sources · PUT .../config/sources	테이블·조회 컬럼·파티션 기준 열
PUT /api/pipeline/config/alert-cols · alert-hint	매칭알람 전송 열 · 추천 컨텍스트
GET /api/pipeline/alerts · alerts/outbox · PUT alerts/ack	알람 목록 · 업로드 폴더 현황 · ack
GET /api/jobs/state · /stream · POST /enqueue(-all)	백필 plan/chunk 상태 · SSE · 큐 투입
POST /api/jobs/cancel · retry-partition · probe-invalidate	chunk 취소 · 파티션 재실행 · probe 캐시 무효화
GET /api/s3/items · POST /run · /stop · GET /browse-keys	전송 항목 · 실행/중지 · S3 key 탐색
GET /api/scanner/result/{vehicle} · PUT /ignore/{vehicle}	FAB 스캔 결과 · 무시 목록
GET /api/health · /api/version	모드 확인 · VERSION.json

이 리포트의 근거. Valve 저장소 `backend/core/*.py` (20개 모듈) · `backend/routers/*.py` · `config/*.yaml|json` · `app.py` · `VERSION.json` · `README.md` · `docs/flow_integration.md` 를 2026-07-29 시점(v0.3.9 / e3f9e8f)에 직접 읽어 작성했다. 수치·기본값은 코드와 `config/` 시드 값을 그대로 옮긴 것이며, 설치본의 `config/` 는 seed-only 라 **운영 중인 설치에서는 값이 다를 수 있다** — 실제 값은 웹 설정 화면 또는 `GET /api/pipeline/config` 로 확인할 것.